

SQL 처리기 완전정복

크래프톤 정글 WEEK6 — 이것만 보면 수요 코딩회 준비 끝

목차

Chapter 1. C 기초 복습 — 이것만 알면 된다

1-1. 포인터

1-2. 문자열

1-3. 구조체와 열거형

1-4. 파일 입출력

Chapter 2. SQL이 뭔지 이해하기

Chapter 3. 전체 설계 — 큰 그림

Chapter 4. types.h — 데이터 설계

Chapter 5. parser.c — SQL 파싱 (핵심)

Chapter 6. storage.c — 파일 기반 DB

Chapter 7. executor.c — 실행기

Chapter 8. main.c — CLI + 전체 연결

Chapter 9. Makefile — 빌드 자동화

Chapter 10. 테스트 전략

Chapter 11. 엡지 케이스 & 에러 처리

Chapter 12. 차별화 추가 구현

Chapter 13. 발표 준비 체크리스트

C 기초 복습 — 이것만 알면 된다

SQL 처리기를 만들려면 C에서 딱 4가지만 확실히 알면 된다: 포인터, 문자열, 구조체, 파일 입출력. 나머지는 다 이 4가지 조합이다.

1-1. 포인터 — 주소를 가리키는 변수

포인터가 뭐냐? 메모리 주소를 저장하는 변수다. 그게 다다.

```
int a = 42;           // 정수 변수. 값은 42.
int* p = &a;          // 포인터 변수. a의 "주소"를 저장.

printf("%d\n", *p);   // *p = "p가 가리키는 곳의 값" = 42
```

비유하면 이렇다:

a = 아파트 101호에 사는 사람 (값: 42) p = "101호"라고 적힌 메모지 (주소를 저장) *p = 메모지 보고 101호 찾아가서 사람 만남 (값: 42) &a = 101호의 주소를 알려줌

이 프로젝트에서 포인터가 쓰이는 곳

- 1 문자열 처리 — C에서 문자열은 사실 `char*` (char 포인터)다.

```
char* sql = "INSERT INTO students VALUES (1, 'Kim', 20);";

// sql[0] = 'I', sql[1] = 'N', ... 이런 식으로 접근 가능
// sql은 문자열의 "시작 주소"를 가리키는 포인터
```

2 함수에 구조체 넘기기 — 포인터로 넘기면 원본을 직접 수정할 수 있다.

```
// 포인터 없이 넘기면: 복사본이 전달됨 (원본 변경 불가)
void bad(ParsedSQL result) {
    result.type = SQL_INSERT; // 이걸 복사본만 바뀜. 원본은 그대로.
}

// 포인터로 넘기면: 원본 주소가 전달됨 (원본 직접 수정)
void good(ParsedSQL* result) {
    result->type = SQL_INSERT; // 원본이 바뀜!
}

// 호출할 때:
ParsedSQL parsed;
good(&parsed); // &parsed = parsed의 주소를 넘김
```

핵심 규칙 3개만 기억해:

&변수 = 변수의 주소를 꺼낸다

*포인터 = 포인터가 가리키는 값을 꺼낸다

포인터->멤버 = 포인터가 가리키는 구조체의 멤버 접근 ((*포인터).멤버 의 축약)

3 파일 포인터 — FILE* 도 포인터다.

```
FILE* fp = fopen("data.csv", "r");
// fp는 "열린 파일"을 가리키는 포인터
// fgets, fprintf 등의 함수에 이 포인터를 넘겨서 파일 작업
```

1-2. 문자열 — SQL 파싱의 기반

C에는 `string` 타입이 없다. 대신 `char` 배열 + 끝에 `'\0'`이 문자열이다.

```
char name[10] = "Kim";
```

```
// 메모리에 이렇게 저장됨:
```

```
// [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
// 'K' 'i' 'm' '\0' ? ? ? ? ?
//                               ↑ 여기가 끝 표시 (null terminator)
```

이 프로젝트에서 필수인 문자열 함수들

1. strlen() — 문자열 길이

```
strlen("Hello"); // 5 ('\0'은 안 셈)
```

```
strlen("INSERT INTO "); // 12
```

```
// 용도: "INSERT INTO " 다음 위치로 이동할 때
```

```
const char* p = sql + strlen("INSERT INTO ");
```

```
// → sql 시작점에서 12칸 뒤로 이동 = 테이블 이름 시작점
```

2. strcmp() — 문자열 비교

```
// C에서는 == 로 문자열 비교 불가! (주소 비교가 됨)
```

```
if (str == "hello") // ❌ 이러면 안 됨
```

```
if (strcmp(str, "hello") == 0) // ✅ 이렇게 해야 함
```

```
// strcmp 반환값:
```

```
// 0 → 같다
```

```
// 양수 → str이 더 크다 (사전순)
```

```
// 음수 → str이 더 작다
```

3. strncasecmp() — 대소문자 무시하고 N글자 비교

```
// SQL은 대소문자를 안 가림: INSERT = insert = Insert
```

```
strncasecmp("INSERT INTO", "insert into", 11); // 0 (같다)
```

```
strncasecmp("SELECT", "select", 6); // 0 (같다)
```

```
// 용도: SQL 문장이 INSERT로 시작하는지 확인
```

```
if (strncasecmp(sql, "INSERT INTO", 11) == 0) {  
    // INSERT 문이다!  
}
```

4. strchr() — 문자 1개 찾기

```
char* sql = "VALUES (1, 'Kim', 20)";  
  
char* p = strchr(sql, '(');  
// p는 이제 "(1, 'Kim', 20)" 을 가리킴  
// 즉 '('가 있는 위치의 주소를 반환  
  
// 못 찾으면 NULL 반환  
char* q = strchr(sql, '@');  
// q = NULL (@ 없으니까)
```

5. strcasestr() — 부분 문자열 찾기 (대소문자 무시)

```
char* sql = "INSERT INTO students VALUES (1, 'Kim', 20);";  
  
char* p = strcasestr(sql, "VALUES");  
// p는 "VALUES (1, 'Kim', 20);" 을 가리킴  
// "values", "Values" 도 다 찾아줌
```

6. strtok() — 문자열 분리 (매우 중요!)

```
char str[] = "1, 'Kim', 20";  
  
// 첫 번째 호출: 원본 문자열 전달  
char* token = strtok(str, ",");  
// token = "1"  
  
// 두 번째부터: NULL 전달 (이전 위치에서 이어서 분리)  
token = strtok(NULL, ",");  
// token = " 'Kim'"  
  
token = strtok(NULL, ",");
```

```
// token = " 20"

token = strtok(NULL, ",");
// token = NULL (더 이상 없음)
```

strtok 주의사항:

strtok은 원본 문자열을 직접 수정한다! 심표를 '\0'으로 바꿔버림.

그래서 원본을 보존하고 싶으면 복사본을 만들어서 strtok에 넘겨야 한다.

```
char buf[1024];
strcpy(buf, original); // 복사본 만들기
strtok(buf, ",");      // 복사본을 쪼갬 (원본은 안전)
```

7. strncpy() / strcpy() — 문자열 복사

```
char dest[64];

// 전체 복사
strcpy(dest, "hello"); // dest = "hello"

// N글자만 복사 (안전함)
strncpy(dest, source, 10);
dest[10] = '\0'; // 반드시 끝에 '\0' 직접 넣어줘야!
```

8. strcspn() — 특정 문자까지의 길이

```
char line[] = "Hello World\n";
int len = strcspn(line, "\n"); // 11 ('\n' 이전까지의 길이)

// 줄바꿈 제거 트릭 (이 프로젝트에서 매우 자주 씬):
line[strcspn(line, "\n")] = '\0';
// '\n' 위치에 '\0'을 덮어써서 줄바꿈 제거
```

9. isspace() — 공백 문자 판별

```
// 스페이스, 탭, 줄바꿈 등을 공백으로 판별
isspace(' ');    // 참 (1)
isspace('\t');   // 참 (1)
isspace('A');    // 거짓 (0)

// 용도: 앞쪽 공백 건너뛰기
while (isspace(*str)) str++;
// "  hello" → str이 "hello"를 가리키게 됨
```

문자열 함수 요약 치트시트:

길이 알기 → `strlen`

비교하기 → `strcmp` / `strncasecmp`

한 글자 찾기 → `strchr`

부분 문자열 찾기 → `strcasestr`

쪼개기 → `strtok`

복사하기 → `strcpy` / `strncpy`

공백 판별 → `isspace`

1-3. 구조체와 열거형

구조체(struct)는 여러 변수를 하나로 묶는 것이다. Python의 class, JS의 object와 비슷하다.

```
// 정의
typedef struct {
    char name[64];
    int age;
    float gpa;
} Student;
```

```
// 사용
Student s;
strcpy(s.name, "Kim"); // 문자열은 strcpy로 넣기
s.age = 20;             // 숫자는 = 으로 넣기
s.gpa = 3.8;

printf("%s is %d years old\n", s.name, s.age);
// 출력: Kim is 20 years old
```

포인터로 접근할 땐 -> 를 쓴다:

```
Student* p = &s;
printf("%s\n", p->name); // "Kim"
p->age = 21;             // 원본의 age가 21로 바뀜
```

열거형(enum)은 상수에 이름을 붙이는 것이다.

```
// 정의
typedef enum {
    SQL_INSERT,    // 내부적으로 0
    SQL_SELECT,    // 내부적으로 1
    SQL_UNKNOWN    // 내부적으로 2
} SqlType;

// 사용: 숫자 대신 이름으로 쓰니까 코드가 읽기 쉬워짐
SqlType type = SQL_INSERT;

if (type == SQL_INSERT) {
    // INSERT 처리
}

// switch문과 궁합이 좋다:
switch (type) {
    case SQL_INSERT: /* ... */ break;
    case SQL_SELECT: /* ... */ break;
    default: /* 에러 */ break;
}
```


Q. typedef는 뭔가요?

typedef 는 타입에 별명을 붙이는 것이다. typedef 없으면 매번 `struct ParsedSQL` 이라고 써야 하는데, typedef 쓰면 그냥 `ParsedSQL` 이라고만 쓸 수 있다. 편의 기능이다.

1-4. 파일 입출력

이 프로젝트의 절반이 파일 읽기/쓰기다. 패턴이 항상 같으니까 외워두면 좋다.

파일 열기/닫기 기본 패턴

```
// 1. 열기
FILE* fp = fopen("파일경로", "모드");

// 2. 열기 실패 체크 (파일 없으면 NULL 반환)
if (fp == NULL) {
    fprintf(stderr, "파일 열기 실패!\n");
    return -1;
}

// 3. 파일 작업 (읽기 or 쓰기)
// ...

// 4. 닫기 (반드시!)
fclose(fp);
```

파일 열기 모드 (이것만 알면 됨)

모드	의미	이 프로젝트에서의 용도
"r"	읽기 전용	SELECT할 때 CSV 읽기, SQL 파일 읽기

"w"	쓰기 (기존 내용 삭제)	파일 새로 만들 때
"a"	추가 (기존 내용 뒤에 이어쓰기)	INSERT할 때 CSV에 행 추가

파일에서 읽기 — fgets()

```
FILE* fp = fopen("data.csv", "r");

char line[1024]; // 한 줄을 담을 버퍼

// fgets: 한 줄씩 읽음. 파일 끝이면 NULL 반환.
while (fgets(line, sizeof(line), fp) != NULL) {
    // line에 한 줄이 들어있음 (줄바꿈 포함!)
    line[strcspn(line, "\n")] = '\0'; // 줄바꿈 제거
    printf("읽은 줄: %s\n", line);
}

fclose(fp);
```

fgets 동작 원리:

파일에서 한 줄을 읽어서 `line` 배열에 넣는다.

줄바꿈(`\n`)까지 포함해서 읽는다. 그래서 직접 제거해줘야 한다.

`sizeof(line)` 은 버퍼 크기. 이것보다 긴 줄은 잘림 (안전장치).

파일에 쓰기 — fprintf()

```
FILE* fp = fopen("data.csv", "a"); // "a" = 끝에 추가

// printf랑 사용법 동일. 첫 인자가 파일 포인터일 뿐.
fprintf(fp, "%d,%s,%d\n", 1, "Kim", 20);
// 파일에 "1,Kim,20\n" 이 추가됨

fclose(fp);
```

Q. printf vs fprintf 차이는?

`printf` 는 화면(터미널)에 출력. `fprintf` 는 지정한 파일에 출력. `fprintf`의 첫 인자로 `stdout` 을 넘기면 `printf`와 동일하게 동작한다. `stderr` 를 넘기면 에러 출력으로 쓸 수 있다.

CHAPTER 2

SQL이 뭔지 이해하기

SQL(Structured Query Language)은 데이터베이스에 명령을 내리는 언어다. 우리가 구현할 건 이 중에 딱 2개:

INSERT — 데이터 넣기

```
INSERT INTO 테이블이름 VALUES (값1, 값2, 값3);
```

실제 예시:

```
INSERT INTO students VALUES (1, 'Kim', 20);
-- students 테이블에 {id:1, name:'Kim', age:20} 추가

INSERT INTO students VALUES (2, 'Lee', 22);
-- students 테이블에 {id:2, name:'Lee', age:22} 추가
```

이걸 우리 프로그램에서는 CSV 파일에 한 줄 추가로 구현한다:

`students.csv` 파일: | 1,Kim,20 | ← 첫 번째 INSERT 결과 | 2, Lee, 22 | ←
두 번째 INSERT 결과

SELECT — 데이터 조회

```
SELECT * FROM 테이블이름;
```

실제 예시:

```
SELECT * FROM students;
-- students 테이블의 모든 데이터를 보여줘
-- * 은 "전부 다"라는 뜻
```

이걸 우리 프로그램에서는 CSV 파일을 읽어서 화면에 출력으로 구현한다.

SQL 문장의 구조

```
INSERT INTO students VALUES (1, 'Kim', 20);
-- 키워드 테이블명 키워드 값들 SELECT * FROM students;
-- 키워드 컬럼 키워드 테이블명
```

우리가 할 일의 본질:

SQL 문자열에서 키워드 사이사이에 있는 테이블명 과 값들 을 뽑아내는 것. 이게 파싱이다.

CHAPTER 3

전체 설계 — 큰 그림

프로그램 사용 방법

```
$ cat input.sql
INSERT INTO students VALUES (1, 'Kim', 20);
INSERT INTO students VALUES (2, 'Lee', 22);
SELECT * FROM students;
```

```
$ ./sql_processor input.sql
```

```
> INSERT INTO students VALUES (1, 'Kim', 20);
→ OK
> INSERT INTO students VALUES (2, 'Lee', 22);
→ OK
> SELECT * FROM students;
Row 0: 1,Kim,20
Row 1: 2,Lee,22
```

```

| main.c |
|
| argv[1]로 SQL 파일 경로를 받는다 | | 파일을 열고, 한 줄씩 읽으면서 아래 과정을 반복한다 |
|
| SQL 문장 1줄 ▼
|
| parser.c |
|
| "INSERT INTO students VALUES (1, 'Kim', 20);" | | | | → type: SQL_INSERT | | →
| table_name: "students" | | → values: ["1", "Kim", "20"] | | → value_count: 3 | | |
| 이 결과를 ParsedSQL 구조체에 담는다 |
|
| ParsedSQL 구
| 조체 ▼ |
|
| executor.c | | ParsedSQL의 type을 보고 분기: | | SQL_INSERT → storage_insert() 호출 | |
| SQL_SELECT → storage_select() 호출 |
|
| ▼
|
| storage.c |
|
| INSERT: data/students.csv 파일 끝에 "1,Kim,20\n" 추가 | | SELECT: data/students.csv
| 파일을 읽어서 화면에 출력 |

```

`sql_processor/` |— `types.h` 설계도. 구조체/상수 정의. 모든 파일이 이걸 `include`. |— `parser.h`
`parser.c`의 함수 선언 (헤더) |— `parser.c` SQL 문자열 → ParsedSQL 구조체로 변환 |— `storage.h`
`storage.c`의 함수 선언 (헤더) |— `storage.c` 파일 읽기/쓰기 (CSV) |— `executor.h` `executor.c`
의 함수 선언 (헤더) |— `executor.c` 파싱 결과를 받아서 `storage` 함수 호출 |— `main.c` 프로그램 진입
점. CLI + 파일 읽기 + 전체 연결 |— `test.c` 단위 테스트 |— `Makefile` 빌드 자동화 |— `data/` 테이블
파일들이 저장되는 폴더 |— `students.csv` |— `courses.csv`

Q. 헤더 파일(.h)은 왜 필요한가요?

C에서는 함수를 사용하기 전에 "이런 함수가 있다"고 선언해야 한다. 헤더 파일은 이 선언을 모아둔 것이다. 다른
.c 파일에서 `#include "parser.h"` 하면 `parser.c`에 있는 함수를 쓸 수 있게 된다.

비유하면: .h = 메뉴판 (뭐가 있는지 보여줌), .c = 주방 (실제로 만듦)

CHAPTER 4

types.h — 데이터 설계

모든 파일이 공유하는 구조체와 상수를 정의한다. 프로그램의 "설계도".

```
#ifndef TYPES_H           // 중복 include 방지 (관례적으로 항상 씬)
#define TYPES_H

// — 상수 정의 —
#define MAX_COLUMNS      32    // 한 테이블 최대 컬럼 수
#define MAX_VALUE_LEN    256  // 하나의 값 최대 길이
#define MAX_TABLE_NAME   64    // 테이블 이름 최대 길이
#define MAX_SQL_LEN      1024  // SQL 한 줄 최대 길이

// — SQL 종류 —
```

```

typedef enum {
    SQL_INSERT,      // 0
    SQL_SELECT,      // 1
    SQL_UNKNOWN      // 2
} SqlType;

// — 파싱 결과 구조체 —
typedef struct {
    SqlType type;                // 어떤 SQL인지
    char table_name[MAX_TABLE_NAME]; // 테이블 이름

    // INSERT할 때 쓰는 필드
    char values[MAX_COLUMNS][MAX_VALUE_LEN]; // 값 배열
    int value_count;                          // 값 개수

    // SELECT할 때 쓰는 필드
    char columns[MAX_COLUMNS][MAX_VALUE_LEN]; // 컬럼 목록
    int column_count;                          // 컬럼 개수
} ParsedSQL;

#endif

```

ParsedSQL이 채워지는 과정 시각화

SQL: **INSERT INTO students VALUES (1, 'Kim', 20);** ↓ 파싱 후

	ParsedSQL	type = SQL_INSERT
table_name = "students"	values[0] = "1"	values[1] = "Kim"
	values[2] = "20"	value_count = 3
	columns = (사용 안 함)	column_count = 0

SQL: **SELECT * FROM students;** ↓ 파싱 후

	ParsedSQL	type = SQL_SELECT
table_name = "students"	columns[0] = "*"	column_count = 1
(사용 안 함)	value_count = 0	values =

Q. #ifndef / #define / #endif는 뭔가요?

Include Guard라고 부른다. 같은 헤더가 여러 번 include되면 "이미 정의됨" 에러가 난다. 이것 방지하는 관례적인 패턴이다.

```
#ifndef TYPES_H = "TYPES_H가 정의 안 되어 있으면"

#define TYPES_H = "TYPES_H를 정의해" (다음번엔 통과 못 함)

#endif = "조건 끝"
```

CHAPTER 5

parser.c — SQL 파싱 (핵심 중의 핵심)

이 프로젝트에서 가장 어렵고, 가장 중요한 파일이다. 시간을 가장 많이 쓸 곳.

parser.h (헤더 파일)

```
#ifndef PARSER_H
#define PARSER_H

#include "types.h"

// 외부에서 호출할 함수 선언 (메뉴판)
int parse_sql(const char* sql, ParsedSQL* result);

#endif
```

필요한 헤더들

```
#include <stdio.h>    // printf, fprintf
#include <string.h>    // strlen, strcmp, strchr, strtok, strncpy, ...
#include <ctype.h>     // isspace
#include "types.h"
#include "parser.h"
```


보조 함수: trim (공백 제거)

SQL에서 값 사이에 공백이 들어갈 수 있다. `"'Kim'"` → `"'Kim'"` 으로 정리해야 한다.

```
// 문자열 앞뒤 공백을 제거하는 함수
// 원리: 앞쪽은 포인터를 이동, 뒤쪽은 '\0'을 덮어쓰
static char* trim(char* str) {
    // 1단계: 앞쪽 공백 건너뛰기
    while (isspace(*str)) str++;
    //   " hello " → str이 "hello "를 가리킴

    // 빈 문자열이면 바로 리턴
    if (*str == '\0') return str;

    // 2단계: 뒤쪽 공백 제거
    char* end = str + strlen(str) - 1;
    //   end는 마지막 문자를 가리킴
    while (end > str && isspace(*end)) end--;
    //   end가 공백이 아닌 마지막 문자까지 이동
    *(end + 1) = '\0';
    //   그 다음 위치에 '\0' 덮어쓰기 = 문자열 끝 당기기

    return str;
}
```

trim 동작 예시: 입력: `" hello world "` 1단계 (앞쪽 공백 건너뛰기): `[ ][ ][h][e][l][l][o][ ][w][o][r][l][d][ ][ ][\0]` ↑ str이 여기를 가리킴 2단계 (뒤쪽 공백 제거): `[h][e][l][l][o][ ][w][o][r][l][d][\0]` ↑ 여기에 `'\0'` 덮어쓰기 결과: `"hello world"`

Q. static이 함수 앞에 붙으면?

`static` 이 함수 앞에 붙으면 "이 .c 파일 안에서만 쓸 수 있는 함수"라는 뜻이다. 다른 파일에서 호출 불가. trim은 parser.c 안에서만 쓰니까 static을 붙인 것.

```

static int parse_insert(const char* sql, ParsedSQL* result) {
    result->type = SQL_INSERT;

    // — 1단계: 테이블 이름 추출 —————
    //
    // sql = "INSERT INTO students VALUES (1, 'Kim', 20);"
    //      ~~~~~
    //      12글자 (공백 포함)
    //
    // strlen("INSERT INTO ") = 12이므로
    // sql + 12 = "students VALUES (1, 'Kim', 20);"
    //
    const char* p = sql + strlen("INSERT INTO ");

    // p에서 공백이나 '('를 만날 때까지 = 테이블 이름
    int i = 0;
    while (*p && !isspace(*p) && *p != '(') {
        result->table_name[i++] = *p++;
    }
    result->table_name[i] = '\0';
    // result->table_name = "students"

    // — 2단계: VALUES 위치 찾기 —————
    //
    // strcasestr: 대소문자 무시하고 "VALUES" 찾기
    // "values", "Values" 등도 다 찾아줌
    //
    const char* values_pos = strcasestr(sql, "VALUES");
    if (!values_pos) return -1; // VALUES 없으면 에러

    // — 3단계: 괄호 안의 값 추출 —————
    //
    // '(' 찾기
    const char* paren_open = strchr(values_pos, '(');
    if (!paren_open) return -1;
    paren_open++; // '(' 다음 칸으로 이동
    // paren_open = "1, 'Kim', 20);"

    // ')' 찾기
    const char* paren_close = strchr(paren_open, ');');

```

```

if (!paren_close) return -1;

// '('와 ')' 사이 내용을 복사 (strtok이 원본을 변경하니까)
char buf[MAX_SQL_LEN];
int len = paren_close - paren_open;
strncpy(buf, paren_open, len);
buf[len] = '\0';
// buf = "1, 'Kim', 20"

// — 4단계: 쉼표로 분리 —————
//
result->value_count = 0;
char* token = strtok(buf, ",");

while (token != NULL) {
    char* trimmed = trim(token); // 공백 제거

    // 작은따옴표 제거: 'Kim' → Kim
    if (trimmed[0] == '\') {
        trimmed++; // 앞 따옴표 건너뛰기
        trimmed[strlen(trimmed) - 1] = '\0'; // 뒤 따옴표 제거
    }

    strcpy(result->values[result->value_count], trimmed);
    result->value_count++;

    token = strtok(NULL, ","); // 다음 값
}

return 0; // 성공
}

```

INSERT 파싱 단계별 시각화: 원본: "INSERT INTO students VALUES (1, 'Kim', 20);" 1단계 테이블 이름: INSERT INTO [students] VALUES (1, 'Kim', 20); L p가 여기서 시작 → 공백 만날 때까지 읽음 2단계 VALUES 찾기: INSERT INTO students [VALUES] (1, 'Kim', 20); L strcasestr로 위치 찾음 3단계 괄호 안 추출: VALUES ([1, 'Kim', 20]); L buf에 복사됨 4단계 쉼표로 분리: "1, 'Kim', 20" → strtok(",")

1	,	'Kim'	,	20
values[0]		values[1]		values[2]

← trim + 따옴표 제거 후

```

static int parse_select(const char* sql, ParsedSQL* result) {
    result->type = SQL_SELECT;

    // — 1단계: "SELECT "와 "FROM" 사이 = 컬럼 목록 —
    //
    // sql = "SELECT * FROM students;"
    //      ^^^^^^ ^^^^
    //      7글자   FROM 위치
    //
    const char* col_start = sql + strlen("SELECT ");
    // col_start = "* FROM students;"

    const char* from_pos = strcasestr(sql, "FROM");
    if (!from_pos) return -1;

    // col_start부터 from_pos까지 = 컬럼 부분
    char cols_buf[MAX_SQL_LEN];
    int len = from_pos - col_start;
    strncpy(cols_buf, col_start, len);
    cols_buf[len] = '\0';
    // cols_buf = "* " 또는 "name, age " 등

    // — 2단계: 컬럼 파싱 (쉼표로 분리) —————
    //
    result->column_count = 0;
    char* token = strtok(trim(cols_buf), ",");
    while (token) {
        strcpy(result->columns[result->column_count], trim(token));
        result->column_count++;
        token = strtok(NULL, ",");
    }
    // SELECT * → columns[0] = "*", column_count = 1
    // SELECT name, age → columns[0]="name", columns[1]="age"

    // — 3단계: 테이블 이름 추출 —————
    //
    const char* table_start = from_pos + strlen("FROM ");
    while (isspace(*table_start)) table_start++; // 혹시 모를 공백

```

```

int ti = 0;
while (*table_start && !isspace(*table_start) && *table_start != ';' ) {
    result->table_name[ti++] = *table_start++;
}
result->table_name[ti] = '\0';
// table_name = "students"

return 0;
}

```

SELECT 파싱 단계별 시각화: 원본: "SELECT * FROM students;" 1단계 컬럼 추출: SELECT [*] FROM students; L "SELECT "와 "FROM" 사이 2단계 컬럼 분리: "*" → columns[0] = "*" 3단계 테이블 이름: SELECT * FROM [students]; L "FROM " 다음부터 공백이나 ';'까지

메인 파싱 함수 (진입점)

```

// 외부에서 호출하는 유일한 함수
// SQL 문자열을 받아서 ParsedSQL에 결과를 채운다.
// 성공하면 0, 실패하면 -1 반환.

int parse_sql(const char* sql, ParsedSQL* result) {
    // 구조체 초기화 (모든 값을 0으로)
    memset(result, 0, sizeof(ParsedSQL));

    // 앞쪽 공백 건너뛰기
    while (isspace(*sql)) sql++;

    // SQL 종류 판별 후 해당 파서 호출
    if (strncasecmp(sql, "INSERT INTO", 11) == 0) {
        return parse_insert(sql, result);
    }
    if (strncasecmp(sql, "SELECT", 6) == 0) {
        return parse_select(sql, result);
    }

    // 어느 것도 아니면 UNKNOWN

```

```
    result->type = SQL_UNKNOWN;  
    return -1;  
}
```

Q. memset이 뭔가요?

memset(주소, 값, 크기) = 메모리를 특정 값으로 채운다. memset(result, 0, sizeof(ParsedSQL)) 은 result 구조체의 모든 바이트를 0으로 초기화한다는 뜻이다. 이전에 남아있던 쓰레기 값을 깨끗이 지우는 것.

CHAPTER 6

storage.c — 파일 기반 DB

각 테이블을 CSV 파일로 저장한다. INSERT = 파일에 쓰기, SELECT = 파일에서 읽기.

storage.h

```
#ifndef STORAGE_H  
#define STORAGE_H  
  
#include "types.h"  
  
int storage_insert(const char* table_name,  
                  char values[][MAX_VALUE_LEN],  
                  int value_count);  
  
int storage_select(const char* table_name);  
  
#endif
```

storage.c 전체 코드 + 상세 설명

```

#include <stdio.h>
#include <string.h>
#include "types.h"
#include "storage.h"

// 테이블 이름으로 파일 경로 만들기
// "students" → "data/students.csv"
static void get_table_path(const char* table_name, char* path) {
    sprintf(path, "data/%s.csv", table_name);
    // sprintf = 문자열을 "포매팅"해서 path에 저장
    // %s 자리에 table_name이 들어감
}

// — INSERT 구현 —————
int storage_insert(const char* table_name,
                  char values[][MAX_VALUE_LEN],
                  int value_count)
{
    char path[256];
    get_table_path(table_name, path);

    // "a" = append 모드
    // 파일이 있으면: 끝에 추가
    // 파일이 없으면: 새로 생성
    FILE* fp = fopen(path, "a");
    if (!fp) {
        fprintf(stderr, "Error: Cannot open table '%s'\n", table_name);
        return -1;
    }

    // 값들을 쉼표로 이어서 한 줄로 쓰기
    // values = ["1", "Kim", "20"]
    // → 파일에 "1,Kim,20\n" 추가
    for (int i = 0; i < value_count; i++) {
        if (i > 0) fprintf(fp, ","); // 첫 값 빼고 앞에 쉼표
        fprintf(fp, "%s", values[i]);
    }
    fprintf(fp, "\n"); // 줄바꿈 (다음 행과 구분)

    fclose(fp);
}

```

```

    printf("  Inserted into '%s'\n", table_name);
    return 0;
}

// — SELECT 구현 —————
int storage_select(const char* table_name) {
    char path[256];
    get_table_path(table_name, path);

    FILE* fp = fopen(path, "r");
    if (!fp) {
        fprintf(stderr, "  Error: Table '%s' not found\n", table_name);
        return -1;
    }

    char line[MAX_SQL_LEN];
    int row = 0;

    // 한 줄씩 읽으면서 출력
    while (fgets(line, sizeof(line), fp)) {
        line[strcspn(line, "\n")] = '\0'; // 줄바꿈 제거
        printf("  Row %d: %s\n", row++, line);
    }

    if (row == 0) {
        printf("  (empty table)\n");
    }

    fclose(fp);
    return 0;
}

```

storage 동작 시각화: `storage_insert("students", ["1","Kim","20"], 3)`

data/students.csv 파일:  | (기존 내용) | | ... | | **1, Kim, 20** | ← 이 줄
 이 추가됨  `storage_select("students")` data/students.csv 파일에서 한 줄
 씩 읽어서:  | 1, Kim, 20 | → | Row 0:
 1, Kim, 20 | | 2, Lee, 22 | → | Row 1: 2, Lee, 22 | | 3, Park, 21 | → | Row 2:
 3, Park, 21 |  (파일) (화면 출력)

executor.c — 실행기

parser와 storage를 연결하는 다리 역할. 가장 짧고 간단하다.

executor.h

```
#ifndef EXECUTOR_H
#define EXECUTOR_H
#include "types.h"
int execute(ParsedSQL* parsed);
#endif
```

executor.c

```
#include <stdio.h>
#include "types.h"
#include "storage.h"
#include "executor.h"

int execute(ParsedSQL* parsed) {
    switch (parsed->type) {

        case SQL_INSERT:
            // ParsedSQL에서 테이블명, 값들, 개수를 꺼내서 storage에 전달
            return storage_insert(
                parsed->table_name,
                parsed->values,
                parsed->value_count
            );

        case SQL_SELECT:
            // 테이블명만 넘기면 됨
            return storage_select(parsed->table_name);
    }
}
```

```

        default:
            fprintf(stderr, "  Error: Unknown SQL type\n");
            return -1;
    }
}

```

Q. 왜 executor를 따로 분리하나요? storage를 직접 호출하면 안 되나요?

지금은 간단해서 직접 호출해도 되지만, 나중에 WHERE 절이나 ORDER BY 같은 기능이 추가되면 executor에서 "파싱 결과를 해석해서 추가 처리"를 하게 된다. 예를 들어 WHERE age > 20이면 storage에서 전체를 읽은 다음 executor에서 필터링하는 식이다. 관심사의 분리(Separation of Concerns)라고 하는 설계 원칙이다.

CHAPTER 8

main.c — CLI + 전체 연결

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>    // system()
#include <ctype.h>     // isspace()
#include "types.h"
#include "parser.h"
#include "executor.h"

int process_file(const char* filename) {
    FILE* fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "Error: Cannot open '%s'\n", filename);
        return -1;
    }

    char line[MAX_SQL_LEN];

    // SQL 파일을 한 줄씩 읽으면서 처리

```

```

while (fgets(line, sizeof(line), fp)) {
    // 1. 줄바꿈 제거
    line[strcspn(line, "\n")] = '\0';

    // 2. 앞쪽 공백 건너뛰기
    char* trimmed = line;
    while (isspace(*trimmed)) trimmed++;

    // 3. 빈 줄 건너뛰기
    if (*trimmed == '\0') continue;

    // 4. 주석 건너뛰기 (-- 로 시작하는 줄)
    if (strncmp(trimmed, "--", 2) == 0) continue;

    // 5. SQL 문장 출력 (뭘 실행하는지 보여주기)
    printf("\n> %s\n", trimmed);

    // 6. 파싱
    ParsedSQL parsed;
    if (parse_sql(trimmed, &parsed) != 0) {
        fprintf(stderr, " Parse error!\n");
        continue; // 에러 나도 다음 줄 계속
    }

    // 7. 실행
    if (execute(&parsed) != 0) {
        fprintf(stderr, " Execution error!\n");
    }
}

fclose(fp);
return 0;
}

// — 프로그램 진입점 —————
int main(int argc, char* argv[]) {

    // argc = 인자 개수, argv = 인자 배열
    // ./sql_processor input.sql
    // argc = 2
    // argv[0] = "./sql_processor"

```

```

// argv[1] = "input.sql"

if (argc < 2) {
    printf("Usage: %s <sql_file>\n", argv[0]);
    printf("Example: %s input.sql\n", argv[0]);
    return 1;
}

// data 폴더 생성 (없으면 만들, 있으면 무시)
system("mkdir -p data");

return process_file(argv[1]);
}

```

Q. argc, argv가 뭔가요?

프로그램을 실행할 때 커맨드라인에서 넘기는 인자들이다.

`./sql_processor input.sql` 이라고 실행하면:

`argc` = 2 (인자 2개)

`argv[0]` = `./sql_processor` (프로그램 자기 자신)

`argv[1]` = `"input.sql"` (우리가 넘긴 파일 경로)

`argc < 2`이면 파일을 안 넘긴 거니까 사용법을 알려주고 종료.

Q. continue가 뭔가요?

반복문에서 `continue` 를 만나면 남은 코드를 건너뛰고 다음 반복으로 넘어간다. 빈 줄이나 주석을 만났을 때 파싱/실행을 건너뛰고 다음 줄을 읽으라는 뜻이다.

Makefile — 빌드 자동화

매번 `gcc main.c parser.c executor.c storage.c -o sql_processor` 이렇게 치기 귀찮으니까, `make` 한 단어로 빌드하게 해주는 파일이다.

```
CC = gcc                                # 사용할 컴파일러
CFLAGS = -Wall -Wextra -g              # 컴파일 옵션
                                         #   -Wall: 경고 다 보여줘
                                         #   -Wextra: 추가 경고도
                                         #   -g: 디버깅 정보 포함

TARGET = sql_processor                 # 만들 실행파일 이름
SRCS = main.c parser.c executor.c storage.c

# 기본 명령: make 치면 이게 실행됨
all: $(TARGET)

# 실행파일 만드는 규칙
$(TARGET): $(SRCS)
    $(CC) $(CFLAGS) -o $@ $^
    # $@ = TARGET (sql_processor)
    # $^ = SRCS (모든 .c 파일들)

# 정리: make clean 치면 실행
clean:
    rm -f $(TARGET)
    rm -rf data/

# 테스트: make test 치면 실행
test: $(TARGET)
    @echo "=== Test Start ==="
    @rm -rf data/
    @mkdir -p data
    @echo "INSERT INTO students VALUES (1, 'Kim', 20);" > /tmp/test.sql
    @echo "INSERT INTO students VALUES (2, 'Lee', 22);" >> /tmp/test.sql
    @echo "INSERT INTO students VALUES (3, 'Park', 21);" >> /tmp/test.sql
    @echo "SELECT * FROM students;" >> /tmp/test.sql
    ./$(TARGET) /tmp/test.sql
    @echo "=== Test Done ==="
```

```
# 단위 테스트
unit_test: parser.c storage.c executor.c test.c
    $(CC) $(CFLAGS) -o test_runner $^
    ./test_runner

.PHONY: all clean test unit_test
```

사용법:

`make` → 빌드
`make test` → 빌드 + 테스트 실행
`make unit_test` → 단위 테스트 실행
`make clean` → 실행파일, 데이터 폴더 삭제

Makefile 주의: 명령어 앞의 들여쓰기는 반드시 탭(Tab)이어야 한다. 스페이스로 하면 에러 남. 에디터 설정 확인 할 것.

CHAPTER 10

테스트 전략

발표할 때 테스트 과정을 보여주면 완성도가 높아 보인다. 테스트는 크게 2종류:

1. 단위 테스트 (test.c)

개별 함수가 제대로 동작하는지 확인.

```
#include <stdio.h>
#include <string.h>
#include <assert.h>    // assert() 매크로
#include "types.h"
```

```
#include "parser.h"
#include "storage.h"

// — 파서 테스트 —————

void test_insert_basic() {
    ParsedSQL r;
    int ret = parse_sql("INSERT INTO users VALUES (1, 'Alice', 25);", &r);

    assert(ret == 0); // 파싱 성공
    assert(r.type == SQL_INSERT); // INSERT로 인식
    assert(strcmp(r.table_name, "users") == 0); // 테이블명 맞춤
    assert(r.value_count == 3); // 값 3개
    assert(strcmp(r.values[0], "1") == 0); // 첫 번째 값
    assert(strcmp(r.values[1], "Alice") == 0); // 두 번째 값
    assert(strcmp(r.values[2], "25") == 0); // 세 번째 값

    printf(" PASS: test_insert_basic\n");
}

void test_select_all() {
    ParsedSQL r;
    int ret = parse_sql("SELECT * FROM products;", &r);

    assert(ret == 0);
    assert(r.type == SQL_SELECT);
    assert(strcmp(r.table_name, "products") == 0);
    assert(strcmp(r.columns[0], "*") == 0);
    assert(r.column_count == 1);

    printf(" PASS: test_select_all\n");
}

void test_case_insensitive() {
    ParsedSQL r;
    // 소문자로 써도 동작해야 함
    int ret = parse_sql("insert into users values (1, 'Bob', 30);", &r);
    assert(ret == 0);
    assert(r.type == SQL_INSERT);
    printf(" PASS: test_case_insensitive\n");
}
```

```

void test_unknown_sql() {
    ParsedSQL r;
    int ret = parse_sql("DROP TABLE users;", &r);
    assert(ret == -1);           // 파싱 실패해야 함
    assert(r.type == SQL_UNKNOWN); // UNKNOWN이어야 함
    printf(" PASS: test_unknown_sql\n");
}

// — 스토리지 테스트 —————

void test_insert_and_select() {
    // 테스트용 데이터 삭제 후 시작
    system("mkdir -p data && rm -f data/test_table.csv");

    char v1[][MAX_VALUE_LEN] = {"1", "TestName", "99"};
    int ret = storage_insert("test_table", v1, 3);
    assert(ret == 0);

    // 파일이 생성되었는지 확인
    FILE* fp = fopen("data/test_table.csv", "r");
    assert(fp != NULL);

    char line[256];
    fgets(line, sizeof(line), fp);
    line[strcspn(line, "\n")] = '\0';
    assert(strcmp(line, "1,TestName,99") == 0);
    fclose(fp);

    // 정리
    system("rm -f data/test_table.csv");
    printf(" PASS: test_insert_and_select\n");
}

// — 메인 —————

int main() {
    printf("=== Parser Tests ===\n");
    test_insert_basic();
    test_select_all();
    test_case_insensitive();
}

```



```

test_unknown_sql();

printf("\n=== Storage Tests ===\n");
test_insert_and_select();

printf("\n✓ All tests passed!\n");
return 0;
}

```

Q. assert()가 뭔가요?

assert(조건) = 조건이 참이면 아무 일도 안 일어남. 거짓이면 프로그램이 즉시 멈추고 에러 메시지 출력. 테스트 할 때 "이 값이 이거여야 한다"를 검증하는 가장 간단한 방법이다.

2. 기능 테스트용 SQL 파일

```

-- test_basic.sql (기본 동작 확인)
INSERT INTO students VALUES (1, 'Kim', 20);
INSERT INTO students VALUES (2, 'Lee', 22);
INSERT INTO students VALUES (3, 'Park', 21);
SELECT * FROM students;

-- 다른 테이블도 테스트
INSERT INTO courses VALUES (101, 'Database', 3);
INSERT INTO courses VALUES (102, 'Algorithm', 3);
SELECT * FROM courses;

```

```

-- test_edge.sql (엣지 케이스 확인)
insert into students values (4, 'Choi', 23);
SELECT * FROM nonexistent_table;
SELECT * FROM students;

```

엣지 케이스 & 에러 처리

이것들을 처리하면 완성도가 확 올라간다.

케이스	예시	처리 방법
공백 많음	<code>INSERT INTO t VALUES (1 , 'a');</code>	<code>trim()</code> 으로 공백 제거
대소문자 혼합	<code>insert Into t values (1);</code>	<code>strncasecmp/strcasestr</code> 사용
세미콜론 없음	<code>SELECT * FROM t</code>	<code>';</code> 없어도 파싱되게 구현
빈 줄	(아무것도 없는 줄)	<code>main.c</code> 에서 <code>continue</code> 로 건너뛴
주석	<code>-- 이것은 주석</code>	<code>"--"</code> 로 시작하면 건너뛴
없는 테이블 <code>SELECT</code>	<code>SELECT * FROM ghost;</code>	<code>fopen</code> 실패 → 에러 메시지
빈 테이블 <code>SELECT</code>	(데이터 없는 테이블)	<code>"(empty table)"</code> 출력
지원 안 하는 SQL	<code>DROP TABLE t;</code>	<code>SQL_UNKNOWN</code> → 에러 메시지

에러 처리 원칙:

에러가 나도 프로그램이 죽지 않고 다음 줄을 계속 처리해야 한다. `main.c`의 `continue` 가 이 역할을 한다. 한 줄이 실패해도 나머지 SQL은 정상 실행되어야 한다.

차별화 추가 구현

기본을 다 했으면 아래에서 골라서 추가 구현하면 다른 팀과 차이를 만들 수 있다.

추천 1: WHERE 절 (가성비 최고)

```
SELECT * FROM students WHERE age > 20;
```

구현 방법: storage_select에서 전체 행을 읽되, WHERE 조건에 맞는 행만 출력. ParsedSQL에 where_column, where_operator, where_value 필드를 추가하면 된다.

추천 2: DELETE

```
DELETE FROM students WHERE id = 1;
```

구현 방법: 파일을 읽으면서 조건에 안 맞는 행만 임시 파일에 쓰고, 원본을 임시 파일로 교체.

추천 3: 특정 컬럼 SELECT

```
SELECT name, age FROM students;
```

구현 방법: 스키마 정보(컬럼 순서)를 파일 첫 줄에 저장하고, 해당 컬럼 인덱스만 출력.

추천 4: 예쁜 테이블 출력

기존:

Row 0: 1, Kim, 20

Row 1: 2, Lee, 22

개선:

```
+-----+-----+-----+
| id | name | age |
+-----+-----+-----+
| 1 | Kim | 20 |
```

```
| 2 | Lee | 22 |
+---+-----+-----+
2 rows selected.
```

이건 비교적 쉬운데 보여주기(데모) 효과가 크다.

CHAPTER 13

발표 준비 체크리스트

목요일 발표 4분 + QnA 1분. README.md 기준으로 발표.

README.md에 넣을 내용

SQL Processor

C로 구현한 파일 기반 SQL 처리기

실행 방법

```
```bash
make
./sql_processor input.sql
```
```

지원하는 SQL

- INSERT INTO table VALUES (...)
- SELECT * FROM table

아키텍처

(전체 흐름도)

테스트

```
```bash
make test # 기능 테스트
make unit_test # 단위 테스트
```

...

### ## 추가 구현

- (WHERE, DELETE 등 추가한 거)

## 데모 시나리오 (4분)

- 1 프로젝트 소개 (30초) — 뭘 만들었는지, 아키텍처 간단 설명
- 2 기본 데모 (1분 30초) — INSERT 여러 개 → SELECT로 확인 → 파일 열어서 CSV 내용 보여주기
- 3 테스트 데모 (1분) — make test 실행 → 단위 테스트 결과 보여주기 → 엣지 케이스 처리 보여주기
- 4 추가 구현 / 핵심 코드 설명 (1분) — 차별화 포인트 데모 + 파싱 로직 핵심 설명

### 발표 때 꼭 기억할 것:

"AI가 작성한 코드를 완벽히 내 것으로 소화해 그 원리를 설명할 수 있어야 합니다."

QnA에서 "이 코드가 뭘 하는 건가요?" 질문이 나올 수 있다. 특히 parser.c의 파싱 로직은 확실히 이해해야 한다.

## QnA 대비 질문들

### Q. strtok은 어떻게 동작하나요?

첫 호출 시 원본 문자열을 받고, 구분자(쉼표)를 null 문자로 바꿔서 첫 토큰을 반환한다. 이후 NULL을 넘기면 이전 위치에서 이어서 다음 토큰을 반환한다. 원본 문자열이 변경되기 때문에 복사본을 사용했다.

### Q. 왜 CSV 포맷을 선택했나요?

가장 단순하고 사람이 읽을 수 있으며, fgets로 한 줄씩 읽고 strtok으로 쉼표로 분리하면 바로 파싱이 가능해서 구현이 간단하다.

#### Q. 파일에 직접 저장하는 방식의 한계는?

DELETE나 UPDATE 시 전체 파일을 다시 써야 하고, 동시 접근이 안 되며, 데이터가 많아지면 SELECT가 느려진다 (매번 전체 읽기). 실제 DB는 인덱스, 페이지 단위 관리, 트랜잭션 등으로 이 문제를 해결한다.

#### Q. fopen의 "a" 모드와 "w" 모드 차이는?

"a"(append)는 기존 내용을 유지하고 끝에 추가한다. "w"(write)는 기존 내용을 전부 지우고 새로 쓴다. INSERT는 기존 데이터를 보존해야 하니까 "a"를 썼다.

---

이 문서의 모든 코드를 이해하면 수요 코딩회 준비 끝.

화이팅!